DÉVELOPPEMENT DES APPLICATIONS RÉPARTIES

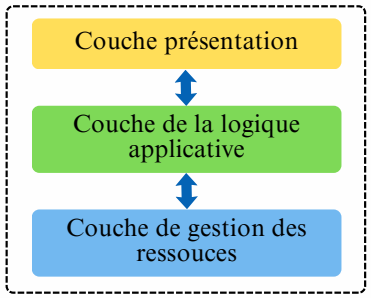
Audience: LI3

Enseignant: Dr. Marwa Thabet
Maître assistant @ISIMM

Réflexions & attentes ?

1. Les systèmes d'information

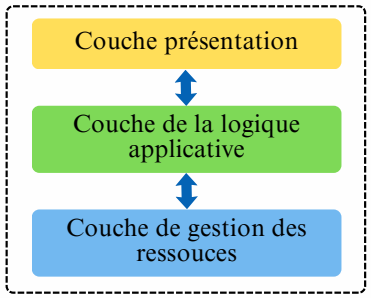
- **Organisation**



- La **couche de présentation** a pour rôle de présenter les informations aux entités externes (utilisateurs humains ou autres ordinateurs) et de leur offrir un moyen d'interagir avec le système, en soumettant des opérations et en recevant des réponses.

1. Les systèmes d'information

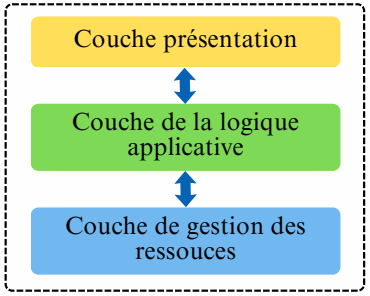
- **Organisation**



- La **logique applicative** veille à exécuter/mener les différentes tâches/opérations du système. Elle peut être: programme, contraintes, règles, processus métiers.

1. Les systèmes d'information

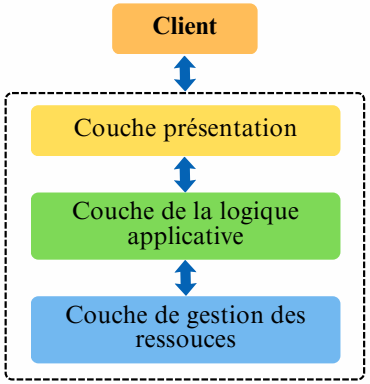
• Organisation



• Le **gestionnaire de ressource** est responsable sur le stockage, l'indexation et l'interrogation des donnés. ça peut être tout système de gestion de données permettant la persistance et l'interrogation des données: une base de données, système de gestion de données textuelles.

1. Les systèmes d'information

• Organisation

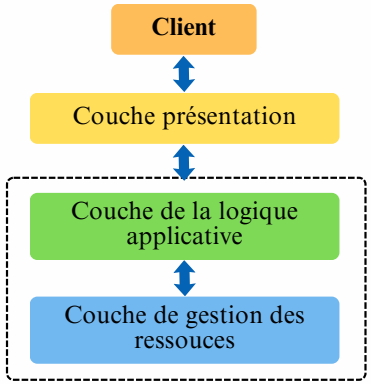


- Un **système monolithique (1-tier)** qui intègre les 3 couches.
- Les utilisateurs/programmes accèdent au système via des terminaux (simple affichage).
- C'est l'architecture des premiers systèmes mainframes.
- **Avantages:**

- **Pas de changement de contexte :** le flot de contrôle reste linéaire et simple, ce qui rend l'exécution rapide.
- **Gestion des ressources centralisée :** facile à contrôler et à administrer puisque tout est regroupé dans une seule machine.
- **Conception optimisée :** comme il n'y a qu'un seul bloc logiciel, on peut optimiser finement ses performances.

1. Les systèmes d'information

• Organisation



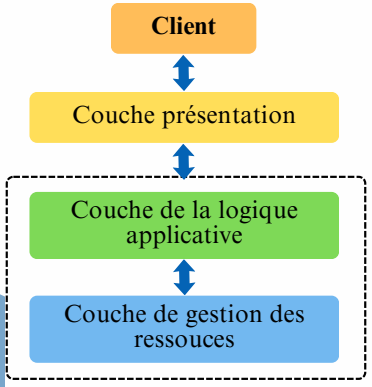
- Un système **client/serveur (2-tiers)** qui déplace la couche de présentation au client.
- Les utilisateurs/programmes accèdent au système via des terminaux (simple affichage).
- C'est l'architecture des premiers systèmes mainframes.
- **Avantages:**

- Les clients sont indépendants.
- Prend en compte les capacités de calcul des clients.
- Introduit le concept de API (Application Program Interface). Une interface pour invoquer le système de l'extérieur.
- Le gestionnaire de ressource voit seulement un seul client: une seule couche applicative.

➡ **Bonne performance puisque il y a pas de connexions/session des clients à maintenir.**

1. Les systèmes d'information

• Organisation



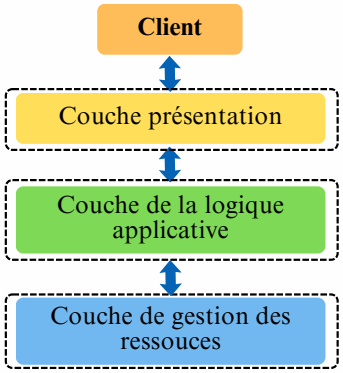
- **Limites:**

- Si un client veut accéder à 2 serveurs ou plus, l'architecture 2 tiers présente les problèmes suivants:
  - Les systèmes sous-jacents ne se connaissent pas
  - Il n y a pas une logique métier commune
- Le client est le point d'intégration (client de plus en plus lourd)
- La responsabilité de gérer l'hétérogénéité des systèmes est transmises aux clients.
- Le client devient responsable de savoir les localisations des entités, comment les utiliser et comment assurer la cohérence.
- Ceci est inefficace de plusieurs point de vue (conception logicielle portabilité et réutilisation de code, performance).

➡ **Ces problèmes sont inhérents à l'architecture 2-tiers.**

1. Les systèmes d'information

- **Organisation**



Pour une architecture 3-tiers:

- Les trois couches sont complètement séparées.
- Typiquement, les couches sont aussi distribuées.
- Modularité de la conception
- Logique applicative pas nécessairement centralisée.
- Les architectures 3-tiers font généralement intervenir une couche intergiciel (middleware).
- Les même problèmes que dans les systèmes 2-tiers.

Chacun de ces tiers n'est pas seulement un logiciel, mais il s'exécute réellement comme un **processus** sur une machine.

2. Rappel sur les processus

Un processus

- **Définition:** entité dynamique représentant l'exécution d'un programme sur une machine.
- **Caractéristiques:**
  - Dispose de ressources allouées (CPU, mémoire, périphériques).
  - Possède un état d'exécution (données en mémoire, registres, pile).
- **Exécution:**
  - Sur une machine multi-CPU, plusieurs processus peuvent s'exécuter en parallèle.
  - Sur une machine mono-CPU, ils s'exécutent en concurrence (ordonnancement par le système).

2. Rappel sur les processus

Parallélisme

- **Définition:** exécution simultanée de plusieurs processus (ou activités) sur plusieurs processeurs.
- **Condition :** possible uniquement sur une architecture multi-CPU.
- **Exemple :**
  - Une mémoire partagée.
  - Un ensemble de processeurs exécutant différents processus en même temps.

2. Rappel sur les processus
Exécution concurrente de processus

- **Systèmes concurrents:** deux ou plusieurs processus (activités) coopèrent pour accomplir une tâche. La coopération possible via :
  - Variables partagées
  - Échange de messages

Processeur P1	Processeur P2
(1) x ← 1	(a) y ← 1
(2) t1 ← y	(b) t2 ← x

2. Rappel sur les processus
Exécution concurrente de processus

- **Concurrence:**
  - Abstraction du parallélisme
  - Exécution alternée (entrelacement) des processus sur un ou plusieurs processeurs.
  - ≠ Parallélisme : les processus ne s'exécutent pas toujours en même temps, mais donnent l'illusion du simultané.

Processeur P1	Processeur P2
(1) x ← 1	(a) y ← 1
(2) t1 ← y	(b) t2 ← x

2. Rappel sur les processus
Exécution concurrente de processus

Processeur P1	Processeur P2
(1) x ← 1	(a) y ← 1
(2) t1 ← y	(b) t2 ← x

- **Instruction atomique :** exécution indivisible, effectuée sans interruption.
- **Processus séquentiel :** enchaînement ordonné d'instructions atomiques.
- **Système concurrent :** composé de plusieurs processus séquentiels s'exécutant de façon potentiellement entrelacée.

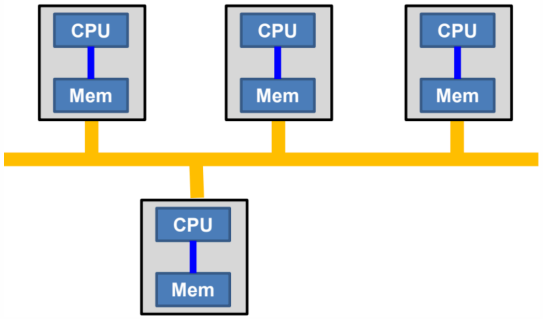
2. Rappel sur les processus
Exécution concurrente de processus

Modèle de l'entrelacement

- On raisonne sur des exécutions séquentielles obtenues par l'entrelacement des instructions des différentes activités.
- Exécution d'un système concurrent = entrelacement arbitraire des instructions atomiques des processus séquentiels.
- Cet entrelacement doit toujours respecter l'ordre interne de chaque processus.

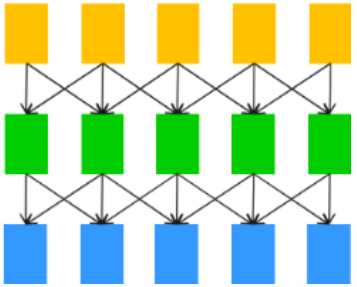
Deux activités $P = p_1; p_2$ et $Q = q_1; q_2$. L'exécution concurrente $P \| Q$ est vue comme (équivalente à) l'une des exécutions :
$p_1; p_2; q_1; q_2$ ou $p_1; q_1; p_2; q_2$ ou $p_1; q_1; q_2; p_2$ ou $q_1; p_1; p_2; q_2$ ou $q_1; p_1; q_2; p_2$ ou $q_1; q_2; p_1; p_2$

- Un **système distribué** est un ensemble de processus exécutés sur des machines distinctes, qui communiquent entre eux via un réseau et coopèrent afin d'atteindre un objectif commun.



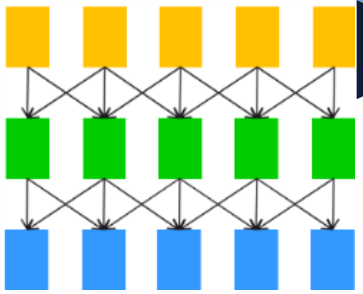
Interactions dans les SID

- Comment connecter les différentes couches et modules d'un SID?
- Comment les différentes parties du système interagissent à travers des interactions entre les modules logicielles?



Communication inter processus (IPC)

- Permet la communication entre des processus
- Transférer des données
- Invoquer une action dans un autre processus (éventuellement s'exécutant dans une autre machine distante).



Contenu du module

1. Programmation réseaux avec les sockets
2. Applications Client/ Serveur
3. Intergiciels orientés objets/ Intergiciels orientés messages
4. Problèmes fondamentaux de la répartition

Chapitre 1

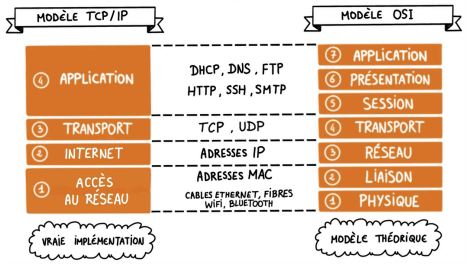
Programmation réseaux avec les sockets

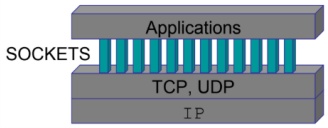
Plan

1. Concept de socket
2. Socket TCP
3. Socket UDP
4. API Java.net.*

1. Concept de socket

- La **notion de sockets** a été introduite dans les distributions Berkeley UNIX (BSD), d'où l'appellation courante de sockets BSD.
- Elles se situent juste au-dessus de la couche transport du modèle OSI (protocoles TCP ou UDP), laquelle repose elle-même sur les services de la couche réseau (protocole IP).





1. Concept de socket

- Le protocole IP est aujourd'hui le plus utilisé pour la communication entre machines. Toutefois, il fournit un service d'envoi de paquets non fiable (sans garantie de livraison ni d'ordre). Il est donc généralement complété par des protocoles de transport :

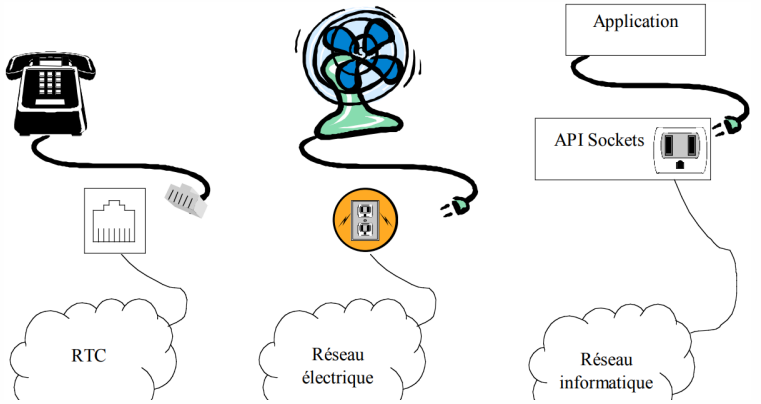
- TCP : offre un service de communication fiable, orienté connexion, avec contrôle de flux et correction d'erreurs.

- UDP : fournit un service non fiable basé sur l'envoi de datagrammes, mais plus léger et plus rapide que TCP.

1. Concept de socket

- Les protocoles TCP/IP et UDP/IP sont accessibles aux applications au moyen des sockets.
- Un socket joue le rôle de point d'accès entre une application et les services fournis par les couches TCP et UDP.
- Ce concept est similaire à l'abstraction de fichier dans UNIX : de la même manière qu'un fichier permet à un programme d'accéder au système de stockage, un socket permet à une application d'accéder au réseau.

1. Concept de socket



1. Concept de socket

- Les sockets offrent :

- Un mécanisme de communication bidirectionnel interprocessus dans un environnement distribué.

- Une API (Application Programming Interface) permettant aux programmeurs d'interfacer leurs applications avec les protocoles de communication sous-jacents.

- **Remarque**: L'interface des sockets est indépendante de toute pile de protocoles particulière. Dans le cadre de ce cours, nous nous focaliserons sur leur utilisation avec les protocoles TCP/IP et UDP/IP.

1. Concept de socket

En fonction du protocole de transport utilisé, les sockets adoptent un mécanisme de communication différent.

Mode connecté (TCP)

- Comparable à un appel téléphonique
- Connexion durable entre les processus
- Communication fiable, orientée flux
- Basé sur le modèle client/serveur

Mode non connecté (UDP)

- Comparable à un envoi de courrier
- Adresse de destination requise à chaque envoi
- Pas d'accusé de réception
- Communication par datagrammes indépendants
- Supporte la multidiffusion (communication vers un groupe de machines)

2. Sockets TCP

Le modèle client/serveur (TCP)

- TCP offre un service connecté et fiable, avec livraison des données dans l'ordre.
- La connexion est asymétrique à l'établissement :
  - Serveur : attend les demandes de connexion.
  - Client : initie la connexion.
- Après connexion, le fonctionnement devient symétrique.
- Deux types de sockets côté serveur :
  - Socket d'écoute : reçoit les demandes de connexion.
  - Socket de service : communication avec un client (un par client connecté).
- Un serveur peut être connecté simultanément à plusieurs clients.

2. Sockets TCP

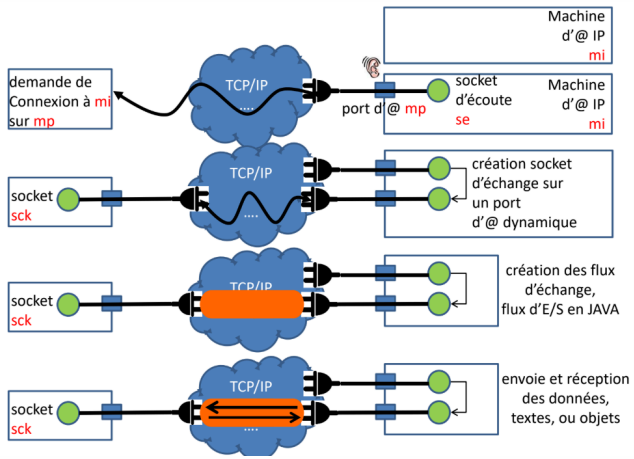
Client

3. Demande de connexion sur la machine d'adresse **mi** et le port **mp** avec le scoket d'échange de flux **sck**
5. création des flux pour échanger des octets, de texte, ou d'objets via **sck**
6. échange via les flux crées
7. fermer le socket **sck**

- Serveur

1. Créer un socket d'écoute **se** sur la machine d'adresse IP **mi** et le port **mp**
2. se mettre en attente / écoute de demande de connexions sur **se**
4. réception d'une demande de connexion ➔ création de socket d'échange sckServ
5. création des flux pour échanger des octets, de texte, ou d'objets via sckServ
6. échange via les flux créés
7. fermer le socket sckServ
8. fermer le socket se

2. Sockets TCP



2. Sockets TCP

Création de socket

• Coté serveur

/* Créer un socket d'écoute se sur la machine d'adresse IP **mi** et le port **mp** */

ServerSocket se = new ServerSocket(mp);

/* se mettre en attente / écoute de demande de connexions sur se */

Socket sckServ = se.accept();

• Coté client

/* Demande de connexion sur la machine d'adresse **mi** et le port **mp** avec le socket d'échange de flux sck */

Socket sck = new Socket (mi , mp);

2. Sockets TCP

Flux d'échange

- **Etant donné un socket sck**

```
InputStream is = sck.getInputStream();
int nb = is.read();
```

```
OutputStream os= sck.getOutputStream();
int res = nb*3;
os.write(res);
```


2. Sockets TCP

Flux d'échange

- **Etant donné un socket sck**

```
InputStream is = sck.getInputStream();
InputStreamReader isr = new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
String str=br.readLine();
```

```
OutputStream os= sck.getOutputStream();
PrintWriter pw = new PrintWriter(os,true);
String rep="Bonjour Mr"+str;
pw.println(rep);  //os.write(res);
```


2. Sockets TCP

Flux d'échange

- Échange d'objets en Java : **Sérialisation / Désérialisation**
  - Sérialisation: Transformation d'un objet (en mémoire) en une suite d'octets transmissibles sur le réseau.
  - Désérialisation: Processus inverse : octets reçus → recréation de l'objet original.
  - Condition : l'objet doit être sérialisable
  - La classe doit implémenter l'interface **Serializable**.

2. Sockets TCP

Flux d'échange

- **Etant donné un socket sck**

```
OutputStream os = sck.getOutputStream();
ObjectOutputStream oos = new ObjectOutputStream(os);
Compte cpt = new Compte("l200", 1000);
oos.writeObject(cpt);
```

```
InputStream is = sck.getInputStream();
ObjectInputStream ois = new ObjectInputStream(is);
Compte cptRecu = (Compte) ois.readObject();
```

2. Sockets TCP

Exemple 1 de communication via des sockets

/* Serveur mono-client */

```
import java.net.*;
import java.io.*;
public class Serveur1 {
public static void main(String[] args){
try {
ServerSocket ss = new ServerSocket(1234);
System.out.println("J'attends une connexion");
Socket s = ss.accept();
InputStream is=s.getInputStream();
OutputStream os=s.getOutputStream();
System.out.println("J'attends un nombre");
int nb=is.read();
int res=nb*3;
System.out.println("J'envoies la réponse");
os.write(res);
s.close();
} catch (Exception e){ e.printStackTrace(); } } }
```

/*Client */

```
import java.net.*; import java.io.*;
import java.util.Scanner;
public class Client1 {
public static void main(String[] args){
try {
Socket s = new Socket("localhost", 1234);
InputStream is=s.getInputStream();
OutputStream os=s.getOutputStream();
Scanner clavier = new Scanner(System.in);
System.out.println("Donner un nombre");
int nb=clavier.nextInt();
os.write(nb);
int res = is.read();
System.out.println("Res ="+res);
os.write(res);
s.close();
} catch (Exception e){ e.printStackTrace(); } } }
```


2. Sockets TCP

Exemple 2 de communication via des sockets

/* Serveur mono-client */

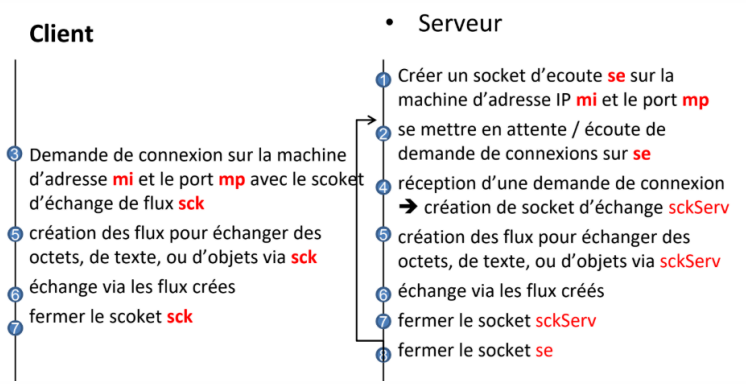
```
import java.net.*; import java.io.*;
public class Serveur2 {
public static void main(String[] args){
try {ServerSocket ss = new ServerSocket(1234);
System.out.println("J'attends une connexion");
Socket s = ss.accept();
InputStream is=s.getInputStream();
InputStreamReader isr = new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
OutputStream os=s.getOutputStream();
PrintWriter pw = new PrintWriter(os,true);
System.out.println("J'attends une chaine");
String str=br.readLine(); //int nb=is.read();
String rep="Bonjour Mr"+str;
System.out.println("J'envoies la réponse");
pw.println(rep); //os.write(res);
s.close();
} catch (Exception e){ e.printStackTrace(); } } }
```

/*Client */

```
import java.net.*;
import java.io.*; import java.util.Scanner;
public class Client2 {
public static void main(String[] args){
try { Socket s = new Socket("localhost", 1234);
InputStream is=s.getInputStream();
InputStreamReader isr = new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
OutputStream os=s.getOutputStream();
PrintWriter pw = new PrintWriter(os,true);
Scanner clavier = new Scanner(System.in);
System.out.println("Donner votre nom");
String str=clavier.next();
pw.println(str);
String res = br.readLine();
System.out.println("Res ="+res);
} catch (Exception e){ e.printStackTrace(); } } }
```


2. Sockets TCP

Exemple 3 de communication via des sockets – Serveur multi-clients itératifs



2. Sockets TCP

Exemple 3 de communication via des sockets – Serveur multi-clients itératifs

```
import java.io.*;
public class Serveur3 {
public static void main(String[] args){
try { ServerSocket ss = new ServerSocket(123);
while(true){
System.out.println("J'attends une connexion");
Socket s = ss.accept();
InputStream is=s.getInputStream();
InputStreamReader isr = new
InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
OutputStream os=s.getOutputStream();
PrintWriter pw = new PrintWriter(os,true);
System.out.println("J'attends une chaine");
String str=br.readLine();
String rep="Bonjour Mr"+str;
System.out.println("J'envoies la réponse");
pw.println(rep);
s.close();
} } catch (Exception e){ e.printStackTrace(); }
} }
```

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads

- **Problème :**

Un serveur classique (mono-thread) ne peut traiter qu'un seul client à la fois → les autres doivent attendre.

- **Solution :**

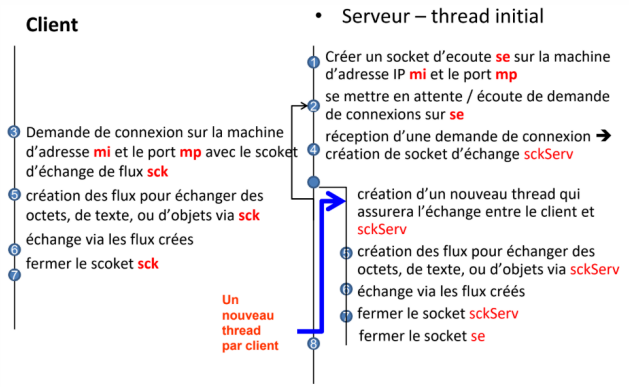
Utiliser le multi-threading pour gérer plusieurs clients simultanément.

- **Principe**

1. Le serveur principal reste en attente (accept()) de nouvelles connexions.
2. À chaque nouvelle connexion :
   - Un nouveau thread est créé.
   - Ce thread est associé à la socket du client.
3. Ce thread dédié gère les opérations d'entrée/sortie (read/write) avec le client.
4. Le serveur principal continue à accepter d'autres connexions.

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads



2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads

```
import java.net.*; import java.io.*;
public class ServeurMT extends Thread{
@Override
public void run(){
try {
ServerSocket ss = new ServerSocket(123);
while(true){
System.out.println("J'attends une connexion");
Socket s = ss.accept();
new Service(s,nbClients).start();
}
} catch (Exception e){ e.printStackTrace(); }
}
public static void main(String[] args){
new ServeurMT().start();
}}
```

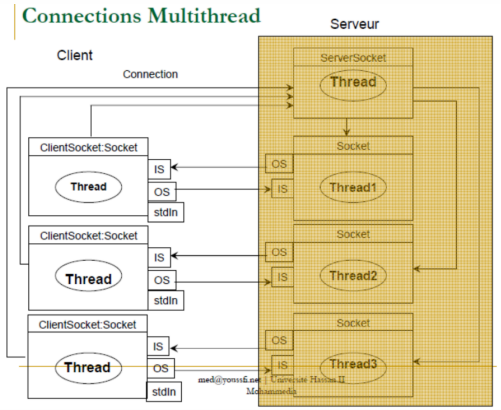

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads

```
import java.net.*; import java.io.*;
public class Service extends Thread {
private Socket s;
private int numClient;
public Service(Socket socket, int numClient){
super(); this.s = socket; this.numClient = numClient; }
public void run(){
try{
InputStream is=s.getInputStream();
InputStreamReader isr = new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
OutputStream os=s.getOutputStream();
PrintWriter pw = new PrintWriter(os,true);
pw.println("Voues êtes client numero"+numClient);
System.out.println("Connexion du client Num "+ numClient);
System.out.println("IP = "+s.getRemoteSocketAddress());
while(true){ String req=br.readLine(); pw.println(req.length());
}
} catch (Exception e){ e.printStackTrace(); }
}
}
```

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads



3. Sockets UDP

- Protocole UDP (User Datagram Protocol)
- **Caractéristiques:**

- Service non connecté : pas d'établissement de connexion préalable.

- Non fiabilisé : les données peuvent être perdues, dupliquées, ou arriver dans un ordre différent.

- Symétrique : chaque processus crée un socket qu'il peut utiliser pour envoyer ou recevoir des datagrammes.

3. Sockets UDP

processus 1 sur machine d'@ IP aip

- Création du DatagramPacket **dgPRecu** dans lequel on recevra le Datagram envoyé
- Création d'un DatagramSocket **dgSck** écoutant des requêtes sur le port numéro **np**
- Réception du DatagramPacket envoyé dans dgPRecu

- processus 2
  - Créer un DatagramPacket dgP qui contient : Les données à envoyer, l'adresse IP aip de la machine cible, et le numéro du port np de la socket vers laquelle ces données seront envoyées.
  - Création d'un DatagramSocket dgS
  - Envoie de dgP via dgS

3. Sockets UDP

- **Créer un DatagramPacket**

```
String message = "Bonjour le monde!" ;
byte[] tampon = message.getBytes();
InetAddress adresse = null;
// recupère l'adresse IP de la machine
adresse = InetAddress.getByName("toto.isimm.rnu.tn");
// crée l'objet qui stockera les données du datagramme à envoyer
DatagramPacket dgp= new DatagramPacket(tampon,tampon.length,adresse,50000);
```

3. Sockets UDP

- **Création d'un DatagramSocket et envoie de dgp**

```
// crée un socket UDP (le port est choisi par le système)
DatagramSocket socket=new DatagramSocket();
// envoie le datagramme UDP
socket.send(dgp);
```


3. Sockets UDP

- **Création du DatagramPacket dgPRecu**

```
byte[] tampon = new byte[1000];
// crée un objet pour stocker les données du datagramme attendu
DatagramPacket dgPRecu = new DatagramPacket(tampon, tampon.length);
// attends puis récupère les données du datagramme
```


3. Sockets UDP

- **Réception du DatagramPacket envoyé dans dgPRECU**

```
// crée un socket UDP qui attends des datagrammes sur le port 50000
DatagramSocket dgSck = new DatagramSocket(50000);
// attends puis récupère les données du datagramme
dgSck.receive(dgpRecu);
// récupère la chaîne de caractère reçue
// Note: dgpRecu.getLength() contient le nombre d'octets reçus
String texte=new String(tampon, 0, dgpRecu.getLength());
System.out.println("Reception de la machine "+
dgpRecu.getAddress().getHostName()+ " sur le port " +dgpRecu.getPort()+" :\n"+ texte );
```

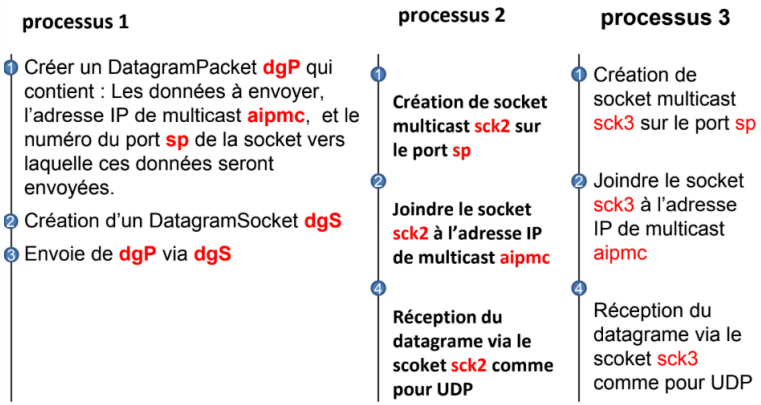

3. Sockets UDP

La classe MulticastSocket

- Cette classe permet d'utiliser le multicasting IP pour envoyer des datagrammes UDP à un ensemble de machines repéré grâce à une addresse multicast (classe D dans IP version 4 : de 224.0.0.1 à 239.255.255.255).
- Les constructeurs de cette classe sont identiques à ceux de la classe **DatagramSocket**.

3. Sockets UDP

Etapes de base de communication via des sockets multicast



3. Sockets UDP

Etapes de base de communication via des sockets multicast

- Création de socket multicast sck2 sur le port sp

```
// crée le socket utilisé pour émettre et recevoir les datagrammes
// il utilisera le port 4321
int sp = 4321;
MulticastSocket sck2 = new MulticastSocket(sp);
```

- Joindre le socket sck2 à l'adresse IP de multicast aipmc

```
// on traveillera avec l'adresse multicast 228.5.6.7
InetAddress groupe = InetAddress.getByName("228.5.6.7");
// s'abonne à l'adresse IP multicast
sck2.joinGroup(groupe);
```

- Le reste des opérations est similaire à l'envoie/réception des paquets UDP

3. Sockets UDP

Etapes de base de communication via des sockets multicast

- Création de socket multicast sck2 sur le port sp

```
// crée le socket utilisé pour émettre et recevoir les datagrammes
// il utilisera le port 4321
int sp = 4321;
MulticastSocket sck2 = new MulticastSocket(sp);
```

- Joindre le socket sck2 à l'adresse IP de multicast aipmc

```
// on traveillera avec l'adresse multicast 228.5.6.7
InetAddress groupe = InetAddress.getByName("228.5.6.7");
// s'abonne à l'adresse IP multicast
sck2.joinGroup(groupe);
```

- Le reste des opérations est similaire à l'envoie/réception des paquets UDP

4. API Java.net.*

La classe InetAddress

- La classe java.net.InetAddress permet de représenter les adresses IP.
- Chaque objet de cette classe possède deux champs :
- **hostName :** contient une chaîne de caractères représentant le nom de l'hôte.
- **address :** contient un tableau d'octets représentant l'adresse IP correspondante.
- Cette classe ne possède pas de constructeur publics.

4. API Java.net.*

La classe InetAddress

- Pour créer un objet de type InetAddress il faut donc utiliser l'une des méthodes suivantes :

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public static InetAddress	getByName (String nom_hote)	Cette méthode utilise le DNS pour renvoyer une instance de la classe InetAddress représentant l'adresse Internet de la machine de nom nom_hote. Lorsque la recherche DNS n'aboutit pas, une exception UnknownHostException est levée.
public static InetAddress[ ]	getAllByName (String nom_hote)	Cette méthode renvoie toutes les adresses Internet de la machine de nom nom_hote.
public static InetAddress	getLocalHost ()	Renvoie une instance de la classe InetAddress représentant l'adresse Internet de la machine locale.
public String	getHostName ()	Renvoie le nom de la machine hôte, ou bien l'adresse IP si la machine n'a pas de nom.
public byte []	getAddress ()	Renvoie l'adresse IP stockée par une instance de la classe InetAddress sous la forme d'un tableau d'octets rangés dans l'ordre standard du réseau (network byte order). Ainsi l'octet d'indice 0 contient l'octet de poids fort de l'adresse.

4. API Java.net.*

La classe InetAddress

- Des méthodes redéfinies:

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public int	hashCode()	• renvoie un code de hachage unique pour chaque adresse. • La méthode hashCode() est utilisée par la HashTable statique de la classe InetAddress. • Cela permet de mémoriser les adresses déjà résolues. • Une adresse demandée plusieurs fois sera extraite du cache → accès plus rapide sans requête DNS.
public boolean	equals(Object obj)	compare un objet InetAddress avec un autre objet.
public String	toString()	renvoie une chaîne de caractères descriptive de l'adresse Internet.

4. API Java.net.*

La classe Socket

- La classe Socket représente en Java les sockets utilisés côtés client ou les sockets de service.

a) Les constructeurs:

SIgnatures	Description
public Socket (String hote, int port) throws UnknownHostException, IOException	• Ce constructeur crée un socket TCP et tente de se connecter sur le port indiqué de l'hôte visé. • Le premier paramètre de ce constructeur représente le nom de la machine serveur. • Si l'hôte est inconnu ou que le serveur de noms de domaine est inopérant, le constructeur générera une UnknownHostException. • Les autres causes d'échec, qui déclenchent l'envoi d'une IOException sont multiples : machine cible refusant la connexion sur le port précisé ou sur tous les ports, problème lié à la connexion Internet, erreur de routage des paquets... **Socket leSocket = new Socket("www.isimm.rnu.tn", 80)**
public Socket (InetAddress addresse, int port) throws IOException	• Ce constructeur fonctionne comme le premier, mais prends comme premier paramètre une instance de la classe **InetAddress**. • Ce constructeur provoque une **IOException** lorsque la tentative de connexion échoue mais il ne renvoie pas d'UnknownHostException puisque cette information est connue lors de la création de l'objet InetAddress.

4. API Java.net.*

La classe Socket

- La classe Socket représente en Java les sockets utilisés côtés client ou les sockets de service.

a) Les constructeurs:

SIgnatures	Description
public Socket(String hote, int port, InetAddress addresseLocale, int portLocal) throws IOException	• Comme les deux précédents, ce constructeur crée un socket et tente de se connecter sur le port indiqué de l'hôte visé. Le numéro du port et le nom de la machine sont fournis dans les deux premiers paramètres et la connexion s'établit à partir de l'interface réseau physique (choix d'une carte réseau sur un système possédant plusieurs accès réseau) ou virtuelle (système multi-adresse) et du port local indiqués dans les deux derniers paramètres. • Si portLocal vaut 0, la méthode (comme les deux premières d'ailleurs) choisit un port disponible (parfois appelé port anonyme) compris entre 1024 et 65 535. • Comme la première méthode cette méthode peut générer une IOException en cas de problème de connexion et lorsque le nom d'hôte donné n'est pas correct (il s'agit dans ce cas d'une unknownHostException).
public Socket(InetAddress address, int port, InetAddress addresseLocale, int portLocal) throws IOException	• Cette méthode est identique à la précédente à la seule différence que le premier paramètre est, comme pour la seconde méthode, un objet InetAddress.

4. API Java.net.*

La classe Socket

b) Méthodes informatives:

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public InetAddress	getInetAddress ()	Ces méthodes renvoient l'adresse Internet et le port distants auquel le socket est connecté.
public int	getPort ()	
public InetAddress	getLocalAddress ()	Ces méthodes renvoient l'adresse Internet et le port locaux que le socket utilise.
public int	getLocalPort ()	

4. API Java.net.*

La classe Socket

c) Méthodes pour communication avec un socket:

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public InputStream	getInputStream () throws IOException	• Cette méthode renvoie un flux d'entrées brutes grâce auquel un programme peut lire des informations à partir d'un socket. • Il est d'usage de lier cet InputStream à un autre flux offrant d'avantage de fonctionnalités (un DataInputStream par exemple) avant d'acquérir les entrées. DataInputStream fluxEnEntree = new DataInputStream(leSocket.getInputStream());
public OutputStream	getOutputStream () throws IOException	• Cette méthode renvoie un flux de sortie brutes grâce auquel un programme peut écrire des informations sur un socket. • Il est d'usage de lier cet OutputStream à un autre flux offrant d'avantage de fonctionnalités (un DataOutputStream par exemple) avant d'émettre des données. DataOutputStream fluxEnSortie = new DataOutputStream(leSocket.getOutputStream());
public void	close() throws IOException	• Bien que Java ferme tous les sockets ouverts lorsqu'un programme se termine ou bien lors d'un « garbage collect », il est fortement conseillé de fermer explicitement les sockets dont on n'a plus besoin à l'aide de la méthode close. • Une fois un socket fermé on peut toujours utiliser les méthodes informatives, par contre toute tentative de lecture ou écriture sur les « input/output streams » provoque une IOException.

4. API Java.net.*

La classe DatagramPacket

a) Les constructeurs:

SIgnatures	Description
public DatagramPacket(byte[] tampon, int longueur)	• Ce constructeur crée un objet utilisé pour recevoir les données contenues dans des datagrammes UDP reçus par la machine. • Le paramètre tampon doit correspondre à un tableau d'octets (type byte en java) correctement créé (attention si le tableau n'est pas assez grand les données en excès seront détruites). • Le paramètre longueur indique la longueur du tableau. byte[] tampon = new byte[1024]; DatagramPacket datagramme = new DatagramPacket(tampon, tampon.length);
public DatagramPacket (byte[] tampon, int longueur, InetAddress adresse, int port)	• Ce constructeur crée un objet utilisé pour envoyer un datagramme UDP. • Le paramètre tampon doit correspondre aux données à envoyer. • Le paramètre longueur doit indiquer la longueur des données à envoyer. • Le paramètre adresse et le paramètre port doivent indiquer respectivement l'adresse IP et le port de destination. String message = "Bonjour le monde!"; byte[] tampon = message.getBytes(); InetAddress adresse = InetAddress.getByName("www.isimm.rnu.tn"); DatagramPacket datagramme = new DatagramPacket(tampon, tampon.length, adresse, 7);

4. API Java.net.*

La classe DatagramPacket

b) Les accesseurs

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public byte[]	getData()	Ces méthodes permettent de récupérer et de changer le tableau d'octets utilisé soit pour recevoir soit pour envoyer un datagramme UDP.
public void	setData(byte[] buf)	
public void	setLength(int length)	Ces méthodes permettent de récupérer et de changer la longueur du tableau d'octets utilisé soit pour recevoir soit pour envoyer un datagramme UDP.
public int	getLength()	
public InetAddress	getAddress()	Ces méthodes permettent de récupérer et de changer les adresses IP et numéros de ports. Si l'objet est utilisé pour envoyer des datagrammes UDP, il s'agira de l'adresse et du port de destination sinon, il s'agira de l'adresse et du port de l'émetteur du datagramme UDP.
public void	setAddress(InetAddress iaddr)	
public int	getPort()	
public void	setPort(int iport)	

4. API Java.net.*

La classe DatagramPacket

b) Les accesseurs

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public byte[]	getData()	Ces méthodes permettent de récupérer et de changer le tableau d'octets utilisé soit pour recevoir soit pour envoyer un datagramme UDP.
public void	setData(byte[] buf)	
public void	setLength(int length)	Ces méthodes permettent de récupérer et de changer la longueur du tableau d'octets utilisé soit pour recevoir soit pour envoyer un datagramme UDP.
public int	getLength()	
public InetAddress	getAddress()	Ces méthodes permettent de récupérer et de changer les adresses IP et numéros de ports. Si l'objet est utilisé pour envoyer des datagrammes UDP, il s'agira de l'adresse et du port de destination sinon, il s'agira de l'adresse et du port de l'émetteur du datagramme UDP.
public void	setAddress(InetAddress iaddr)	
public int	getPort()	
public void	setPort(int iport)	

4. API Java.net.*

La classe DatagramSocket

a) Les constructeurs:

Signatures	Description
public DatagramSocket() throws SocketException	Ce constructeur crée un socket UDP qui permet d'envoyer et recevoir des datagrammes UDP. Le port qu'utilise ce socket est choisi par le système d'exploitation. De même l'adresse IP qu'utilise ce socket est choisie automatiquement par le système.
public DatagramSocket(int port) throws SocketException	Ce constructeur crée un socket UDP qui permet d'envoyer et recevoir des datagrammes UDP. Le port qu'utilise ce socket est indiqué par le paramètre port. L'adresse IP qu'utilise ce socket est choisie automatiquement par le système.
public DatagramSocket(int port, InetAddress adresse) throws SocketException	Ce constructeur crée un socket UDP qui permet d'envoyer et recevoir des datagrammes UDP. Le port qu'utilise ce socket est indiqué par le paramètre port. L'adresse IP qu'utilise ce socket est indiqué par le paramètre adresse.

- Ces trois constructeurs peuvent générer des exceptions de type SocketException si il y a un problème de configuration réseau, si le port choisi est déjà utilisé ou si l'adresse choisie n'est pas l'une des adresses de la machine locale.

4. API Java.net.*

La classe DatagramSocket

b) Emission et réception de Datagrammes:

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public void	send(DatagramPacket p) throws IOException	Envoie un datagramme UDP qui contiendra toutes les informations référencées par l'objet p.
public void	receive(DatagramPacket p) throws IOException	Attends un datagramme UDP et lors de sa réception stocke ses informations dans l'objet p. Des exceptions de type IOException peuvent être générées si un problème d'entrée/sortie survient.

4. API Java.net.*

La classe DatagramSocket

c) Méthodes informatives:

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public InetAddress	getLocalAddress ()	Ces méthodes renvoient l'adresse Internet et le port locaux que le socket utilise.
public int	getLocalPort ()	

4. API Java.net.*

La classe MulticastSocket

- Les constructeurs de cette classe sont identiques à ceux de la classe DatagramSocket.
- Pour pouvoir recevoir des datagrammes UDP envoyés grâce au multicasting IP, il faut s'abonner à une adresse multicast (de classe D pour IP version 4).
- De même lorsqu'on ne souhaite plus recevoir des datagrammes UDP envoyés à une adresse multicast on doit indiquer la résiliation de l'abonnement.

4. API Java.net.*

La classe MulticastSocket

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public void	joinGroup(InetAddress adresseMulticast) throws IOException	• Cette méthode permet de s'abonner à l'adresse multicast donnée. • Un même socket peut s'abonner à plusieurs adresses multicast simultanément. D'autre part il n'est pas nécessaire de s'abonner à une adresse multicast si on veut juste envoyer des datagrammes à cette adresse. • Une **IOException** est générée si l'adresse n'est pas une adresse multicast ou si il y a un problème de configuration réseau.
public void	leaveGroup(InetAddress adresseMulticast) throws IOException	• Cette méthode permet de résilier son abonnement à l'adresse multicast donnée. • Une IOException est générée si l'adresse n'est pas une adresse multicast, si la socket n'était pas abonnée à cette adresse ou si il y a un problème de configuration réseau.

4. API Java.net.*

La classe MulticastSocket

- Dans les datagrammes IP se trouve un champ spécifique appelé TTL (Time To Live – durée de vie) qui est normalement initialisé à 255 et décrémenté par chaque routeur que le datagramme traverse.
- Lorsque ce champ atteint la valeur 0 le datagramme est détruit et une erreur ICMP est envoyé à l'émetteur du datagramme. Cela permet d'éviter que des datagrammes tournent infiniment à l'intérieur d'un réseau IP.
- Le multicasting IP utilise ce champ pour limiter la portée de la diffusion. Par exemple avec un TTL de 1 la diffusion d'un datagramme est limitée au réseau local.

4. API Java.net.*

La classe MulticastSocket

Visibilité et type de retour	Nom de la méthode et paramètres	Description
public int	getTimeToLive() throws IOException	Ces méthodes permettent la consultation et la modification du champ TTL qui sera écrit dans les datagrammes envoyés par ce socket.
public void	setTimeToLive(int ttl) throws IOException	

MERCI DE VOTRE ATTENTION